

Informe de Auditoría del Sistema Web

Integrantes del grupo:

Jorge Illera Rivera

Iker Argulo Galán

Oscar Lijerón Pérez

Bruno Izaguirre Martínez de Marañón

María Bogajo

Ander Javier Corral

Fecha:

09/11/2024

Índice de Contenidos

1	Introducción	3
2	Metodología	3
3	Vulnerabilidades	3
3.1	Rotura de control de acceso	3
3.1.1	Soluciones implementadas	3
3.2	Fallos criptográficos	3
3.2.1	Soluciones implementadas	3
3.3	Inyección	4
3.3.1	Cross-Site Scripting (XSS)	5
3.3.2	Inyección SQL	5
3.3.3	Soluciones implementadas	6
3.4	Diseño inseguro	7
3.4.1	Soluciones implementadas	7
3.5	Configuración de seguridad insuficiente	7
3.5.1	Soluciones implementadas	7
3.6	Componentes vulnerables y obsoletos	7
3.6.1	Soluciones implementadas	7
3.7	Fallos de identificación y autenticación	7
3.7.1	Soluciones implementadas	7
3.8	Fallos en la integridad de datos y software	7
3.8.1	Soluciones implementadas	7
3.9	Fallos en la monitorización de la seguridad	7
3.9.1	Soluciones implementadas	7
4	Conclusiones	7
5	Bibliografía	7
6	Apéndices	8

Índice de Figuras

1	Errores de inyección detectados por ZAP.	4
---	--	---

Índice de Tablas

1 Introducción

Este informe detalla la auditoría de seguridad del sistema web utilizando OWASP ZAP, con el objetivo de identificar y mitigar vulnerabilidades.

2 Metodología

La auditoría se realizó mediante un análisis de ZAP, ejecutando...

3 Vulnerabilidades

3.1 Rotura de control de acceso

3.1.1 Soluciones implementadas

3.2 Fallos criptográficos

En esta sección se comentarán los fallos de seguridad web relacionados con la criptografía o su ausencia.

Hemos identificado y solucionado dos fallos criptográficos. El primer fallo fue almacenar los datos de nuestra base de datos sin cifrar, utilizando únicamente el cifrado incorporado por SQL, el cual se descifra directamente al realizar una consulta. El segundo fallo consistió en almacenar las contraseñas en texto plano.

3.2.1 Soluciones implementadas

Para subsanar el primer error decidimos cifrar datos que almacenamos en nuestra base de datos. Para ello se decidió usar el algoritmo de cifrado simétrico ¹ Advanced Encryption Standar (AES) de 256 bits de longitud. Puesto que este algoritmo se considera seguro.

Para usar este algoritmo tuvimos que generar una clave segura para cifrar y descifrar los datos. Por eso usamos un código de Python ² para generarla usando la librería secrets, que es un modulo que genera números aleatorios con la suficiente entropía para que la clave generada pueda ser considerada criptográficamente fuerte. Una vez generada la clave de secreta la guardamos como una variable de entorno en un archivo .env llamado claveCifrado.env para evitar comprometerla. Después implementamos los algoritmos de cifrado y descifrado en nuestro proyecto. Los datos de cifrado/descifrado juegos (nombre, genero, fecha de lanzamiento, valoración y precio) y de usuarios (nombre, teléfono, DNI y fecha de nacimiento) han sido cifrados. Sin embargo, como al cifrar los datos estos aumentan su número de caracteres hemos tenido que cambiar los tipos de dato y la longitud de los campos de la base de datos para poder

¹Método de cifrado que usa la misma clave para cifrar y descifrar

²Lenguaje de programación orientado a objetos usado principalmente para el desarrollo web y de aplicaciones informáticas.

guardalos.

Para solventar el segundo error decidimos usar un algoritmo resumen para guardar el resumen criptográfico ³ de la contraseña en vez de guardarla como texto plano. Para ello usamos la función `password_hash` a la que le damos como parametros la contraseña a resumir y un una constante de PHP ⁴ (Hypertext Preprocessor) `PASSWORD_BCRYPT` que indica que queremos usar al algoritmo Bcrypt, que es un algoritmo seguro para contraseñas. Además, este algoritmo usa sales ⁵ para mayor seguridad ya que dos usuarios con la misma contraseña tendrán resúmenes criptográficos distintos.

3.3 Inyección

¿Qué es la inyección?

La inyección es una vulnerabilidad de seguridad en la que un atacante introduce datos maliciosos en una aplicación, a menudo mediante campos de entrada o URL, logrando manipular comandos o consultas dentro del sistema. Esto permite ejecutar código no autorizado, acceder a datos sensibles o modificar información sin permisos. Un tipo común de inyección es el SQL Injection, que ocurre cuando consultas SQL maliciosas son ejecutadas por la base de datos, generalmente debido a la falta de validación de entrada en la aplicación.

En esta sección se han identificado dos amenaza de alto riesgo. Las vulnerabilidades detectadas corresponden a una Inyección SQL y a un ataque de Cross-Site Scripting (XSS) Reflejado, como se puede observar en la Figura 1.



Figura 1: Errores de inyección detectados por ZAP.

³Criptograma que consiste en una sucesión de caracteres alfanuméricos que representan la totalidad del contenido original

⁴PHP es un lenguaje de código abierto especialmente adecuado para el desarrollo web y que puede ser incrustado en HTML.[4]

⁵caracteres aleatorios que se le añaden a la contraseña para que sea mas difícil precalcular los resúmenes criptográficos

3.3.1 Cross-Site Scripting (XSS)

La vulnerabilidad CWE-79 corresponde a Cross-Site Scripting (XSS) Reflejado, que permite a un atacante inyectar código malicioso en los parámetros de entrada del sistema, como los campos de formularios o las URL. Cuando un usuario accede a una URL o un formulario que contiene este código inyectado, el navegador del usuario ejecuta dicho código, lo que puede llevar a la manipulación de datos, el robo de cookies, la ejecución de scripts no autorizados, y otros ataques de seguridad.[2]

Causa de la vulnerabilidad

La vulnerabilidad XSS detectada se originaba en el archivo `modify_item.php`, más concretamente en el campo de la fecha de lanzamiento y en otros campos de entrada en el formulario de modificación de un juego. Esto se debía a que los valores de entrada eran reflejados en la respuesta sin la sanitización o validación adecuada, permitiendo a los atacantes inyectar y ejecutar código JavaScript en el navegador del usuario.

Pruebas y comportamiento observado

Sin la sanitización de entrada: OWASP ZAP detectó que ciertos campos permitían la inyección de código JavaScript en los valores de los formularios.

Por ejemplo, en el campo de la fecha, se podía ingresar un valor como

`<script>alert (1) ;</script>`, lo cual provocaba la ejecución de un alert en el navegador.

3.3.2 Inyección SQL

La vulnerabilidad CWE-89 corresponde a la inyección SQL, donde un atacante es capaz de insertar o manipular consultas SQL a través de los datos de entrada, en este caso, el parámetro `id` en la URL.

Causa de la Vulnerabilidad

La vulnerabilidad detectada se originaba en el archivo `show_user.php` y fue causada por la construcción dinámica de consultas SQL sin la adecuada sanitización⁶ ni parametrización de los datos proporcionados por el usuario. En este caso, el valor del parámetro `id` se concatenó directamente a la consulta SQL sin protección, lo que permitió la ejecución de código SQL malicioso.

Pruebas y comportamiento observado

En el error encontrado, la inyección fue posible al modificar el parámetro de la siguiente manera:
`1750' AND '1'='1' --`

Esto permitió a OWASP ZAP modificar la consulta SQL ejecutada en el servidor, manipulando los resultados devueltos y accediendo a información que no debía ser revelada. En particular, se manipuló la consulta para obtener datos utilizando una condición booleana que siempre era verdadera, `'1'='1'`, y luego se comentó el resto de la consulta utilizando `--`, lo que invalidaba el resto de la consulta SQL.

⁶Sanitización se refiere al proceso de limpiar o filtrar los datos de entrada para eliminar cualquier contenido malicioso que pudiera comprometer la seguridad del sistema.[3]

3.3.3 Soluciones implementadas

Para mitigar la vulnerabilidad de XSS se implementaron las siguientes soluciones:

- **Escapado⁷ de salida:**

Se utilizó la función `htmlspecialchars()` en PHP para escapar caracteres especiales (como `¡`, `¿`, `&`, `"`, `'`) en todas las salidas. Esto asegura que el contenido reflejado en el navegador sea tratado como texto y no como código ejecutable.

- **Validación y sanitización de fecha de lanzamiento:**

Se implementó una expresión regular (`preg_match`) para asegurarse de que las fechas ingresadas cumplan con el formato `YYYY-MM-DD`.

- **Sanitización de datos entrantes:**

Para cada valor de entrada recibido a través del formulario, se aplicó `mysqli_real_escape_string()` para evitar que cualquier código especial o malicioso llegue a la base de datos.

- **Validación de Campos Numéricos (rating y price):**

Los valores de `rating` y `price` se convirtieron a tipo `float` usando `floatval()` para asegurarse de que se almacenen y procesan correctamente como valores numéricos.

Para mitigar la vulnerabilidad de Inyección SQL se implementaron las siguientes soluciones:

- **Uso de consultas preparadas:**

Se ha utilizado el enfoque de consultas preparadas con declaraciones `bind_param()`. Esto asegura que los parámetros sean tratados de forma segura, separando los datos del código SQL, y evitando así que cualquier entrada maliciosa pueda modificar la estructura de las consultas.

- **Validación y sanitización de entradas**

Además de las consultas preparadas, se valida y filtra las entradas del usuario para asegurarse de que solo se acepten datos válidos. Por ejemplo, el uso de `intval()` para el parámetro `id` convierte cualquier valor no numérico a un valor seguro (como 0), evitando que se inyecten datos inesperados en las consultas.

Todas estas soluciones también se han implementado en otros archivos que, aunque no presentaban errores en el análisis, tenían vulnerabilidades relacionadas con la inyección que necesitaban ser abordadas.

Resultados de la segunda auditoría

Después de implementar las soluciones descritas, se realizó un segundo análisis con OWASP ZAP. Los resultados fueron positivos: no se detectaron vulnerabilidades de XSS ni otros problemas de inyección.

Conclusiones

Ambas vulnerabilidades fueron una amenaza crítica para la aplicación, ya que podía permitir a los atacantes ejecutar código malicioso en el navegador de los usuarios o insertar consultas SQL

⁷Escapado se refiere al proceso de transformar caracteres especiales en una entrada válida para que no sean interpretados como parte del código ejecutable.[3]

a través de los datos de entrada. Gracias a las soluciones implementadas, las vulnerabilidades se han mitigado de manera efectiva. La aplicación ahora cumple con un estándar de seguridad más alto, protegiendo los datos y la experiencia de los usuarios frente a posibles ataques de inyección de código.

3.4 Diseño inseguro

3.4.1 Soluciones implementadas

3.5 Configuración de seguridad insuficiente

3.5.1 Soluciones implementadas

3.6 Componentes vulnerables y obsoletos

3.6.1 Soluciones implementadas

3.7 Fallos de identificación y autenticación

3.7.1 Soluciones implementadas

3.8 Fallos en la integridad de datos y software

3.8.1 Soluciones implementadas

3.9 Fallos en la monitorización de la seguridad

3.9.1 Soluciones implementadas

4 Conclusiones

5 Bibliografía

References

- [1] owasp.org
Injection
<https://owasp.org/www-community/Injection> (accedido 11/07/2024).
- [2] owasp.org
Cross-Site Scripting (XSS)
<https://owasp.org/www-community/attacks/xss/> (accedido 11/07/2024).
- [3] cwe.mitre.org
Sanitización y escapeo
<https://owasp.org/www-community/attacks/xss/> (accedido 11/07/2024).

- [4] php.net
¿Qué es PHP?
<https://www.php.net/manual/es/intro-what-is.php> (accedido 11/07/2024).

6 Apéndices